

# 使用例

この README は、controllability score の使い方をMATLABのコードで順に確認するためのガイドである。

ex01\_\*.m から ex04\_\*.m までのファイルは、ここで説明する使い方をまとめた形で実行できる参考スクリプトとして置いている。

## リポジトリの取得

まず、ターミナルで任意の作業ディレクトリに移動し、GitHub からリポジトリを clone する。

```
git clone https://github.com/Sato-Sakigake-Controllability-Score/controllability-score.git
cd controllability-score
```

以下では、この controllability-score ディレクトリをリポジトリのルートディレクトリと呼ぶ。

## 前提

- MATLAB R2024a 以降
- Control System Toolbox
- figure や GUI を使うコードでは、MATLAB の GUI 表示環境が必要

サンプルスクリプトを実行する場合は、MATLAB でリポジトリのルートディレクトリに移動し、次のようにルートをパスに追加した状態で呼び出す。

```
addpath(pwd) % リポジトリのルートディレクトリをパスに追加
run("examples/ex01_minimal_bothcs.m")
```

## 1. システム行列を用意する

基本 API は、システム行列  $A$  を受け取ってノードごとの重みを返す。

```
% 簡単な対角行列の例
A = diag([-1.0, -2.0, 0.5, 1.2]);
```

一般には、 $A$  の行と列がノードに対応する正方行列であればよく、次のようなバリエーションが考えられる。

- ネットワークの隣接行列  $A$  をそのままシステム行列にする
- 安定化のために、対角成分(自己ループに対応する)を足して、 $A - cI$  の形にする
- ラプラシアン  $L = D - A$  を計算して、 $-L$  をシステム行列とする

```
adjacency = [ ...
    0.0, 0.8, 0.0; ...
    0.0, 0.0, 0.4; ...
    0.2, 0.0, 0.0];

% 自己ループを加えた例
selfLoopWeight = 2.0;
A1 = adjacency - selfLoopWeight * eye(size(adjacency, 1));

% ラプラシアンの場合
degree = sum(adjacency, 2);
L = diag(degree) - adjacency;
A2 = -L;
```

## 2. VCS と AECS を同時に計算する

VCS と AECS の両方を使う場合は, `bothcs` を使うのが基本である.

```
[pV, pA] = bothcs(A);
```

`pV` と `pA` は, それぞれノードごとの重みベクトルである.

```
disp("VCS weights:");
disp(pV);

disp("AECS weights:");
disp(pA);
```

VCS だけ, または AECS だけが必要な場合は, 個別の関数も使える.

```
pV = vcs(A);
pA = aecs(A);
```

ただし, `bothcs(A)` は, アルゴリズムの最初に  $W_1, \dots, W_n$  を一度に計算し, それをもとに VCS と AECS を計算するため, 両方を用いる場合は, `vcs(A)`, `aecs(A)` を順に呼び出すよりも `bothcs(A)` を呼び出す方が効率的である.

## 3. solver の情報を確認する

計算結果の重みだけでなく, 最適化の状態も確認したい場合は, 追加の出力を受け取ることができる.

```
[pV, pA, infoV, infoA] = bothcs(A);
```

例えば, `infoV` は `CSResult` オブジェクトで, 次のようなプロパティを持つ.

```
infoV =
  ObjectiveValue: 8.5001 % 目的関数値
  Gradient: [4×1 double]
  GradNorm: 8
  StepNorm: 0
  Iterations: 1 % 反復回数
  FuncCount: 3
  Converged: 1
  ExitFlag: 1
  ExitMessage: "Step norm below Tol."
  Algorithm: "ProjectedGradient (Armijo, projection arc)"
  SolverOptions: [1×1 ProjectedGradientSolver]
  ProblemInfo: [1×1 struct]
  Trace: []
```

個別のプロパティの詳細は, [README.md](#) の「4.6 クラス (`CSResult`)」セクションを参照されたい.

また, 個別関数でも, 同じように `info` を受け取れる.

```
[pV, infoV] = vcs(A);
[pA, infoA] = aecs(A);
```

## 4. 有限時間ホライズンを指定する

既定では `T = inf` として扱われるが, 有限時間のスコアを計算したい場合は, 第2引数または `T=...` で終端時刻を指定する.

```
T = 2.0;
[pV, pA] = bothcs(A, T);
```

Name-Value 形式でも同じ指定ができる.

```
[pV, pA] = bothcs(A, T=2.0);
```

## 5. その他のオプションを指定する

前節の有限時間ホライズンの指定だけでなく, その他のオプションも Name-Value 形式で指定できる.

- Gramian 側の主な指定: `Method`, `Steps`, `UseScaling`
- solver 側の主な指定: `MaxIter`, `Tol`, `Verbose`, `StoreTrace`

例えば, オプションをまとめて指定した例を示す.

```
T = 2.0;

[pV, pA, infoV, infoA] = bothcs( ...
    A, T, ...
    Method="integral", ...
    Steps=80, ...
    UseScaling=false, ...
    MaxIter=2000, ...
    Tol=1e-9, ...
    Verbose=true, ...
    StoreTrace=true);
```

オプションの詳細は、[README.md](#) の「4.1.1 通常可制御性スコア用API」セクションを参照されたい。

また、`UseScaling` は  $A$  行列が不安定な場合でも計算できるようにするオプションである。一方で、ジョルダン標準形を計算するため、固有値が重複している場合にはエラーが出る場合があることに注意が必要である。

## 6. サンプルスクリプトを実行する

まとめた例として実行したい場合は、次のスクリプトを使う。README の各節は、これらのスクリプトを読むときの見取り図としても使える。

| File                                  | 内容                                                                            |
|---------------------------------------|-------------------------------------------------------------------------------|
| <code>ex01_minimal_bothcs.m</code>    | <code>vcs</code> , <code>aece</code> s, <code>bothcs</code> の基本と solver 情報の確認 |
| <code>ex02_finite_horizon.m</code>    | 複数の有限時間ホライズン $T$ によるスコア比較                                                     |
| <code>ex03_edge_weight_sweep.m</code> | 1 本のエッジ重みを変えたときのスコア変化                                                         |
| <code>ex04_visualize_scores.m</code>  | GUI で作成した隣接行列から VCS と AECS を計算                                                |